

UNIVERSITAS
GADJAH MADA

Pelatihan Kompetensi RPL Mobile SMK Kelas XII (Level Advanced) Pertemuan I – Data Storages

Irkham Huda, S.Kom., M.Cs



Penyimpanan Data

Menyimpan data

- Shared Preferences—Data primitif pribadi dalam pasangan key-value
- Penyimpanan internal—Data pribadi pada memori perangkat
- Penyimpanan eksternal—Data publik pada perangkat atau penyimpanan eksternal
- Database SQLite—Data terstruktur dalam database pribadi
- Content Providers—Simpan secara pribadi dan sediakan untuk umum

Menyimpan data di luar Android

- Koneksi jaringan—Di web dengan server Anda sendiri
- Pencadangan Awan—Mencadangkan aplikasi dan data pengguna di cloud
- Firebase Realtime Database—Simpan dan sinkronkan data dengan database cloud NoSQL di seluruh klien secara realtime

Sistem File Android

- Penyimpanan eksternal -- Direktori publik
- Penyimpanan internal -- Direktori pribadi hanya untuk aplikasi Anda

Aplikasi dapat menelusuri struktur direktori

Struktur dan operasinya mirip dengan Linux dan java.io

Penyimpanan internal

- Selalu tersedia
- Menggunakan sistem file perangkat
- Hanya aplikasi Anda yang dapat mengakses file, kecuali jika disetel secara eksplisit agar dapat dibaca atau ditulis
- Saat pencopotan aplikasi, sistem menghapus semua file aplikasi dari penyimpanan internal

Penyimpanan luar

- Tidak selalu tersedia, dapat dihapus
- Menggunakan sistem file perangkat atau penyimpanan eksternal fisik seperti kartu SD
- Dapat dibaca dunia, sehingga aplikasi apa pun dapat membaca
- Saat uninstall, sistem tidak menghapus file pribadi ke aplikasi

Kapan menggunakan penyimpanan internal/eksternal

Internal adalah yang terbaik saat

- Anda ingin memastikan bahwa baik pengguna maupun aplikasi lain tidak dapat mengakses file Anda

Eksternal paling baik untuk file yang

- tidak memerlukan pembatasan akses dan untuk
- Anda ingin berbagi dengan aplikasi lain
- Anda mengizinkan pengguna untuk mengakses dengan komputer

Simpan file pengguna di penyimpanan bersama

- Simpan file baru yang diperoleh pengguna melalui aplikasi Anda ke direktori publik tempat aplikasi lain dapat mengaksesnya dan pengguna dapat dengan mudah menyalinnya dari perangkat
- Simpan file eksternal di direktori publik

Penyimpanan internal

Penyimpanan internal

- Menggunakan direktori pribadi (private) hanya untuk aplikasi Anda
- Aplikasi selalu memiliki izin untuk membaca/menulis
- Direktori penyimpanan permanen—getFilesDir()
- Direktori penyimpanan sementara—getCacheDir()

Membuat file

```
File file = new File(
context.getFilesDir(), filename);
```

Gunakan standar java.io operator file atau streams untuk berinteraksi dengan file

Penyimpanan luar

Penyimpanan luar

- Di perangkat atau kartu SD
- Tetapkan izin (permission) di Manifes Android
  - Izin tulis (write) termasuk izin baca (read)

```
<uses-permission
   android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission
    android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Selalu periksa ketersediaan penyimpanan

```
public boolean isExternalStorageWritable() {
    String state = Environment.getExternalStorageState();
    if (Environment.MEDIA_MOUNTED.equals(state)) {
        return true;
    }
    return false;
}
```

Contoh direktori publik eksternal

- DIRECTORY_ALARMS dan DIRECTORY_RINGTONES

  Untuk file audio untuk digunakan sebagai alarm dan nada dering

- DIRECTORY_DOCUMENTS

  Untuk dokumen yang telah dibuat oleh pengguna

- DIRECTORY_DOWNLOADS

  Untuk file yang telah diunduh oleh pengguna

developer.android.com/reference/android/os/Environment.html

Mengakses direktori eksternal publik

1. Dapatkan path getExternalStoragePublicDirectory()
2. Buat file

```
File path = Environment.getExternalStoragePublicDirectory(
            Environment.DIRECTORY_PICTURES);
File file = new File(path, "DemoPicture.jpg");
```

Berapa banyak penyimpanan yang tersisa?

- Jika tidak ada cukup ruang, lempar IOException
- Jika Anda mengetahui ukuran file, periksa ruang
  - getFreeSpace()
  - getTotalSpace().
- Jika Anda tidak tahu berapa banyak ruang yang dibutuhkan
  - coba tangkap IOException

Hapus file yang tidak diperlukan lagi

- Penyimpanan luar

  ```
  myFile.delete();
  ```

- Penyimpanan internal

  ```
  myContext.deleteFile(fileName);
  ```

Jangan hapus file pengguna!

Saat pengguna mencopot pemasangan aplikasi Anda, direktori penyimpanan pribadi aplikasi Anda dan semua isinya akan dihapus

Jangan gunakan penyimpanan pribadi untuk konten milik pengguna!

Misalnya

- Foto diambil atau diedit dengan aplikasi Anda
- Musik yang telah dibeli pengguna dengan aplikasi Anda

SharedPreferences & Basis Data SQLite

Basis Data SQLite

- Ideal untuk pengulangan atau data terstruktur, seperti kontak
- Android menyediakan database mirip SQL
- Tercakup dalam bab dan praktik tersendiri

Shared Preferences

- Membaca dan menulis sejumlah kecil data primitif sebagai pasangan kunci/nilai ke file di penyimpanan perangkat

- Tercakup dalam bab selanjutnya

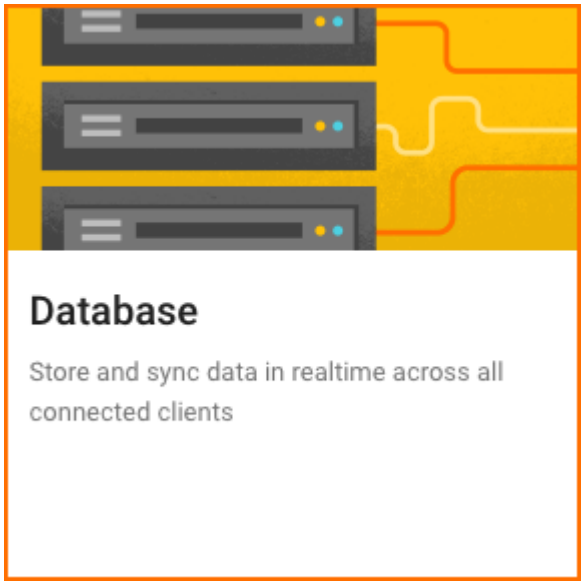
Opsi Penyimpanan Lainnya

Gunakan Firebase untuk menyimpan dan berbagi data

Simpan dan sinkronkan data dengan database cloud Firebase

Data disinkronkan ke semua klien, dan tetap tersedia saat aplikasi Anda offline

firebase.google.com/docs/database/



Basis Data Waktu-Nyata Firebase

- Aplikasi yang terhubung berbagi data
- Dihosting di cloud
- Data disimpan sebagai JSON
- Data disinkronkan secara realtime ke setiap klien yang terhubung



Shared Preferences

Apa itu Shared Preference?

- Baca dan menulis sejumlah kecil data primitif sebagai pasangan kunci/nilai ke file di penyimpanan perangkat
- Kelas SharedPreference menyediakan API untuk membaca, menulis, dan mengelola data ini
- Data bersifat pribadi untuk aplikasi
- Simpan data di onPause()
  pulihkan di onCreate()

Membuat Shared Preferences

- Hanya perlu satu file Shared Preferences per aplikasi
- Beri nama dengan nama paket aplikasi Anda—unik dan mudah dikaitkan dengan aplikasi
- Argumen MODE untuk getSharedPreferences() adalah untuk kompatibilitas mundur—gunakan hanya MODE_PRIVATE agar aman

getSharedPreferences()

```
private String sharedPrefFile =
    "com.example.android.hellosharedprefs";

mPreferences =
    getSharedPreferences(sharedPrefFile,
                         MODE_PRIVATE);
```

Menyimpan Shared Preferences

- SharedPreferences.Editor
- Mengurus semua operasi file
- metode menimpa data (overwrite) jika ada kunci
- apply() menyimpan secara asinkron dan aman

SharedPreferences.Editor

```
@Override
protected void onPause() {
   super.onPause();
   SharedPreferences.Editor preferencesEditor =
       mPreferences.edit();
   preferencesEditor.putInt("count", mCount);
   preferencesEditor.putInt("color", mCurrentColor);
   preferencesEditor.apply();
}
```

Memulihkan Shared Preferences

- Pulihkan di onCreate() di Aktivitas
- Dapatkan metode mengambil dua argumen—kunci, dan nilai default jika kunci tidak dapat ditemukan
- Gunakan argumen default sehingga Anda tidak perlu menguji apakah ada preferensi dalam file

Mendapatkan data di onCreate()

```
mPreferences = getSharedPreferences(sharedPrefFile, MODE_PRIVATE);
mCount = mPreferences.getInt("count", 1);
mShowCount.setText(String.format("%s", mCount));

mCurrentColor = mPreferences.getInt("color", mCurrentColor);
mShowCount.setBackgroundColor(mCurrentColor);

mNewText = mPreferences.getString("text", "");
```

Membersihkan

- Panggil clear() pada SharedPreferences.Editor dan terapkan perubahan

- Anda dapat menggabungkan panggilan untuk menempatkan dan menghapus. Namun, saat Anda apply(), clear() selalu dilakukan terlebih dahulu, apa pun urutannya!

clear()

```
SharedPreferences.Editor preferencesEditor =
                mPreferences.edit();
preferencesEditor.clear();
preferencesEditor.apply();
```

Terima kasih